



**Department of Computer Science and Engineering
University of Dhaka**

**CSE 4113: Internet Programming Lab
Lab 4: Updated Software Requirements Specification (SRS) with
public repository link**

**Project Title:
CSE SmartPortal: A Unified Web, CMS and
Academic Resource Management System**

Team Name: LogicLoop

Submitted To:

Dr. Mamun Or Rashid, Professor, CSEDU
Md. Fahim Arefin, Lecturer, CSEDU
Md. Ahasanul Alam, Lecturer, CSEDU

Submitted By:

Abdullah Al Arman Emon (50)
Shah Jamal Islam (81)
Suhail Tanvir Nahin (82)
Md. Rakib Hossain (88)
Imran Shorif Shuvo

April 14, 2026

Contents

1	Document Control and Public Repository Link	3
1.1	Metadata	3
1.2	Authors and Reviewers	4
1.3	Revision History	4
2	Problem Definition	5
2.1	Problem Statement	5
2.2	Target Users	5
2.3	Goals and Non-Goals	6
3	Scope and Context	7
3.1	In Scope	7
3.2	Out of Scope	8
3.3	Assumptions and Constraints	8
4	Stakeholders and Roles	10
4.1	Stakeholders	10
4.2	User Roles and Permissions	11
5	Functional Requirements	13
6	User Stories and Use Cases	22
6.1	User Stories	22
6.2	Critical Use Cases	24
7	Data Requirements	26
7.1	Core Entities	26
7.2	Data Dictionary	27
7.3	Data Model	28
8	UI and UX Requirements	29
8.1	Wireframes	29
8.2	UX and Accessibility Baseline	30
9	Non-Functional Requirements	31
10	Risks and Open Questions	33
10.1	Risk Register	34
10.2	Open Questions	34
11	Testability and Traceability	36
11.1	Requirement Traceability	36
11.2	Initial Manual Test Plan	37
12	Lab 2 Submission Checklist	39
12.1	Lab 2 Submission Checklist	40
A	API Route Summary	41

B Frontend Route Map	42
C References	43

List of Tables

1.1 Document Metadata	3
1.2 Authors and Reviewers	4
1.3 Revision History	4
4.1 Project Stakeholders	10
4.2 User Roles and Permissions	11
4.3 Definitions, Acronyms, and Abbreviations	12
6.1 UC-01: Student Course Enrolment	24
6.2 UC-02: Admin Creates Exam Schedule	24
6.3 UC-03: Student Submits Finance Payment	25
6.4 UC-04: Visitor Submits Admission Application	25
7.1 Core Data Entities	26
7.2 Data Dictionary (Selected Key Fields)	27
7.3 Implemented Technology Stack	28
10.1 Risk Register	34
11.1 Requirements Traceability Matrix	37
11.2 Initial Manual Test Plan	38
A.1 Implemented API Route Summary	41
B.1 Frontend Route Map	42

Chapter 1

Document Control and Public Repository Link

1.1 Metadata

Field	Value
Document Title	CSE SmartPortal – Software Requirements Specification
Version	2.0
Status	Updated / Released
Date	April 14, 2026
Project	CSE SmartPortal: A Unified Web, CMS and Academic Resource Management System
Repository URL	github.com/team-logicloop/cse-smartportal
Course	CSE 4113: Internet Programming Lab
Institution	Department of Computer Science and Engineering, University of Dhaka

Table 1.1: Document Metadata

1.2 Authors and Reviewers

Role	Name	Remarks
Author	Abdullah Al Arman Emon (50)	Team LogicLoop
Author	Shah Jamal Islam (81)	Team LogicLoop
Author	Suhail Tanvir Nahin (82)	Team LogicLoop
Author	Md. Rakib Hossain (88)	Team LogicLoop
Author	Imran Shorif Shuvo	Team LogicLoop
Supervisor	Dr. Mamun Or Rashid	Professor, CSEDU
Supervisor	Md. Fahim Arefin	Lecturer, CSEDU
Supervisor	Md. Ahasanul Alam	Lecturer, CSEDU

Table 1.2: Authors and Reviewers

1.3 Revision History

Version	Date	Description	Author
1.0	Mar 7, 2026	Initial SRS draft	Team LogicLoop
2.0	Apr 14, 2026	Updated to align with implemented code-base; addressed peer-review gaps (scholarship document upload, exam scheduling, admission workflow, finance, resource hub, course management, meetings)	Team LogicLoop

Table 1.3: Revision History

Chapter 2

Problem Definition

2.1 Problem Statement

The Department of Computer Science and Engineering, University of Dhaka (CSEDU) currently manages its academic operations, communications, and administrative processes through fragmented, manual, or disconnected tools. Faculty announcements, student notices, course materials, exam schedules, admission forms, equipment borrowing records, and fee payments are handled through separate channels – emails, physical notice boards, spreadsheets, or third-party platforms – with no unified interface.

This fragmentation leads to:

- Delayed and inconsistent communication between students, faculty, and administration.
- No centralised digital repository for course resources, assignments, or submission tracking.
- Lack of transparency in exam scheduling, finance obligations, and equipment availability.
- Difficulty in managing admissions, meetings, and RSVP tracking at scale.
- No role-based access control ensuring information is visible only to authorised users.

CSE SmartPortal is proposed as a unified, role-based web platform to consolidate all departmental operations into a single, secure, and maintainable system.

2.2 Target Users

The system targets four distinct user groups:

- **Visitors** – Members of the general public seeking information about CSEDU's programs, people, events, and admission procedures.
- **Students** – Enrolled students who need access to course materials, notices, exam schedules, finance payments, equipment borrowing, and attendance records.

- **Teachers** – Faculty members who create and manage courses, post assignments, track submissions, schedule meetings, and maintain academic profiles.
- **Administrators** – Departmental staff with full platform control including content management, user oversight, admission review, and system configuration.

2.3 Goals and Non-Goals

Goals:

1. Provide a single, authenticated web portal for all departmental academic and administrative activities.
2. Enforce role-based access control so each actor sees only what is relevant to their role.
3. Replace manual, paper-based, or email-driven processes with structured, auditable digital workflows.
4. Ensure all functional modules are aligned with the implemented FastAPI/React codebase.
5. Support scalability from SQLite (development) to PostgreSQL (production) without code changes.

Non-Goals:

1. The system does not replace a full Learning Management System (LMS) such as Moodle or Canvas.
2. The system does not integrate with the University of Dhaka's central student information system.
3. Real-time video conferencing is not implemented; meeting URLs link to external platforms (Zoom, Google Meet).
4. The finance module implements a mock payment gateway only; no real payment processing is included in this version.
5. Mobile native applications (iOS/Android) are out of scope; the web frontend is responsive but not a native app.

Chapter 3

Scope and Context

3.1 In Scope

CSE SmartPortal encompasses the following functional modules, all of which are implemented in the codebase:

- **Authentication & Role Management** – JWT-based registration, login, and password change with roles: Student, Teacher, Admin.
- **Public-Facing Website** – Home page, People Directory, Chairman Profile, Notice Board, Events, Meetings, Admission Hub.
- **Content Management System (CMS)** – Admin-controlled notices, events, and announcements with batch scoping and file attachments.
- **Course Management** – Teacher-created courses with schedule, passcode-based enrolment, classroom interface, assignments, and resource hub.
- **Academic Operations** – Exam scheduling (Midterm/Final), attendance tracking (monthly missing-class percentage).
- **Admission Hub** – Multi-step application form with document upload; admin review and deletion workflow.
- **Departmental Meetings** – Meeting scheduling with URLs; RSVP (Yes/No) tracking per user.
- **Equipment Borrowing** – Lab inventory management; student borrow/return requests with date ranges.
- **Finance Management** – Admin-created fee events; mock payment gateway; admin payment verification.
- **Teacher Profile Management** – Editable profile with biography, research papers, and academic social links.
- **Student Dashboard** – Role-specific view of enrolled courses, notices, exams, and finance obligations.
- **Admin Dashboard** – Central navigation to all management panels.

3.2 Out of Scope

The following are explicitly excluded from this version of CSE SmartPortal:

- Integration with the University of Dhaka’s central ERP or student records system.
- Real-time notifications (push notifications, websockets, email alerts).
- Live video conferencing or streaming within the portal.
- Real payment gateway integration (Bkash, Nagad, card processors).
- Native mobile applications for iOS or Android.
- Automated grading or AI-assisted assignment evaluation.
- Multi-department or multi-institution support.

3.3 Assumptions and Constraints

Assumptions:

- Users have reliable internet access and use a modern web browser (Chrome, Firefox, Edge, Safari).
- The server environment supports Python 3.12 and Node.js for build tooling.
- A relational database engine (SQLite for development; PostgreSQL for production) is available.
- Environment variables (`DATABASE_URL`, `SECRET_KEY`, `RESOURCE_HUB`) are correctly configured before deployment.
- The department will supply initial seed data (equipment inventory, people directory JSON) as needed.
- Students use email addresses with the domain `@cs.du.ac.bd`; teachers and officers use `@cse.du.ac.bd`.

Constraints:

- The system must be implemented as a web-based application accessible from modern browsers.
- Backend API must follow RESTful conventions using FastAPI routers with versioned prefixes (e.g., `/v1/`).
- Authentication must use JWT tokens; all protected routes require a valid bearer token.
- File uploads are stored on the server’s local filesystem and served via a static file mount at `/resources`.
- The project must be maintained in a public Git repository with proper commit history and documentation.

- All API responses must use JSON format; HTTP status codes must conform to REST semantics.
- The frontend must implement role-based route guards (enforced via `RequireAuth` component).

Chapter 4

Stakeholders and Roles

4.1 Stakeholders

Stakeholder	Interest	Influence
CSEDU Administration	Digitise departmental operations; reduce manual processes	High – primary client and decision-maker
Faculty Members (Teachers)	Manage courses, assignments, and student interaction efficiently	High – primary content creators
Students	Access course materials, notices, exams, finance, and borrow equipment	High – primary end users
System Administrator	Maintain platform stability, user accounts, and data integrity	High – full system control
General Public (Visitors)	Access public departmental information and admission procedures	Medium – read-only access
Course Supervisors	Verify project requirements are met for CSE 4113	Medium – academic reviewers

Table 4.1: Project Stakeholders

4.2 User Roles and Permissions

Role	Email Domain	Capabilities	Restrictions
Visitor	None (unauthenticated)	Browse public pages: home, people, chairman, notice board, events, meetings, admission hub	Cannot access any authenticated routes
Student	@cs.du.ac.bd	View courses, notices, exams, finance events; submit assignments and payments; borrow equipment; RSVP meetings; update own profile	Cannot create courses, notices, events, or manage other users
Teacher	@cse.du.ac.bd	Create and manage courses; post assignments and resources; view submissions; grade work; update own profile and research papers; RSVP meetings	Cannot manage system-wide content, users, or admin features
Admin	@cse.du.ac.bd	Full system control: manage notices, events, exams, admissions, equipment inventory, finance events, meetings, attendance records, and user oversight	—

Table 4.2: User Roles and Permissions

Definitions, Acronyms, and Abbreviations:

Term	Description
SRS	Software Requirements Specification
API	Application Programming Interface
JWT	JSON Web Token – used for stateless authentication
CRUD	Create, Read, Update, Delete
ORM	Object-Relational Mapping (SQLAlchemy)
CORS	Cross-Origin Resource Sharing
CMS	Content Management System
RSVP	Répondez s'il vous plaît – meeting attendance response
Admin	System administrator with full platform control
Teacher	Faculty member who manages courses and academic content
Student	Enrolled student who accesses academic resources
Visitor	Unauthenticated user browsing public pages
Batch	Academic cohort identifier (e.g., Batch 27)
Resource Hub	Static file server for course materials
Finance Event	A named fee/payment item with deadline and batch scope

Table 4.3: Definitions, Acronyms, and Abbreviations

Chapter 5

Functional Requirements

FR-AUTH: Authentication and User Management

FR-AUTH-01: The system shall allow any user to register with an email address, password, and role (student or teacher). Upon successful registration, a corresponding student or teacher profile record shall be created automatically.

Acceptance Criteria: POST `/v1/auth/register` returns HTTP 200 with a user object; duplicate email returns HTTP 400.

FR-AUTH-02: The system shall authenticate registered users via email and password, returning a signed JWT access token upon success.

Acceptance Criteria: POST `/v1/auth/login` returns `{access_token, token_type}` with HTTP 200; invalid credentials return HTTP 401.

FR-AUTH-03: The system shall allow an authenticated user to change their password by providing their current password and a new password.

Acceptance Criteria: POST `/v1/auth/password_change` returns HTTP 200 on success; wrong current password returns HTTP 400.

FR-AUTH-04: The system shall enforce role-based access control on all protected API endpoints. Users without the required role shall receive HTTP 403.

Acceptance Criteria: Accessing an admin endpoint with a student token returns HTTP 403.

FR-AUTH-05: The frontend shall implement route guards (`RequireAuth` component) that redirect unauthenticated or unauthorised users to the login page.

Acceptance Criteria: Navigating to `/admin-dashboard` without an admin token redirects to `/login`.

FR-PUBLIC: Public Website

FR-PUBLIC-01: The system shall display a public home page with departmental information, a video banner, and navigation to all public sections.

Acceptance Criteria: Route `/` renders `HomePage` component without authentication.

FR-PUBLIC-02: The system shall provide a People Directory page listing all faculty members, their designations, and contact details, populated from a structured JSON data source.

Acceptance Criteria: Route `/people` renders the directory; all entries from `people.json` are displayed.

FR-PUBLIC-03: The system shall provide an About Chairman page with the chairman's message, photo, and profile information.

Acceptance Criteria: Route `/chairman` renders correctly; chairman photo and text are visible.

FR-PUBLIC-04: The system shall provide a public Notice Board page displaying all published notices accessible without login.

Acceptance Criteria: Route `/notice-board` returns and renders notices from the API.

FR-PUBLIC-05: The system shall provide a public Departmental Meetings page listing scheduled meetings with their dates and meeting URLs.

Acceptance Criteria: Route `/meetings` renders meeting list fetched from `/v1/meetings/list`.

FR-PUBLIC-06: The system shall display departmental events publicly, allowing visitors to view event details including dates, location, image, and registration link.

Acceptance Criteria: Route `/studentEventShow` renders event cards; clicking a card shows full details.

FR-NOTICE: Notice Management

FR-NOTICE-01: The system shall allow administrators to create a notice with a title, subtitle, content body, source category (Chairman, Admin, Student-Club, Department, Central), optional batch filter, and optional file attachments.

Acceptance Criteria: POST `/admin/notice/create` with valid payload returns HTTP 201 and the created notice object.

FR-NOTICE-02: The system shall allow administrators to delete any notice by its ID.

Acceptance Criteria: DELETE `/admin/notice/delete/{id}` returns HTTP 200 and removes the record.

FR-NOTICE-03: The system shall allow students to retrieve notices filtered by their batch number, displaying both batch-specific and department-wide notices.

Acceptance Criteria: GET `/student/notice/list` returns notices where `batch` matches the student's batch or `batch` is null.

FR-NOTICE-04: The system shall support attaching one or more file URLs to a notice, stored as a JSON array.

Acceptance Criteria: A notice created with `attachments: [url1, url2]` returns both URLs when retrieved.

FR-EVENT: Event Management

FR-EVENT-01: The system shall allow administrators to create a departmental event with name, description, start date, end date, location, registration deadline, image URL, video URL, and external registration link.

Acceptance Criteria: POST `/admin/events/create` returns HTTP 200 with the event record.

FR-EVENT-02: The system shall allow administrators to update any field of an existing event.

Acceptance Criteria: PUT `/admin/events/update/{id}` with partial payload updates only provided fields; returns updated record.

FR-EVENT-03: The system shall allow administrators to delete an event by its ID.

Acceptance Criteria: DELETE `/admin/events/delete/{id}` returns HTTP 200.

FR-EVENT-04: The system shall provide a public endpoint to list all events ordered by start date, accessible without authentication.

Acceptance Criteria: GET `/admin/events/list` returns a JSON array of all events.

FR-EXAM: Exam Scheduling

FR-EXAM-01: The system shall allow administrators to create an exam record specifying subject name, date and time, duration (in minutes), batch number, room identifier, and exam type (Midterm or Final).

Acceptance Criteria: POST `/admin/exams/create` returns HTTP 201 with the exam object; missing required fields return HTTP 422.

FR-EXAM-02: The system shall allow administrators to delete an exam record by its ID.

Acceptance Criteria: DELETE `/admin/exams/delete/{id}` returns HTTP 200; non-existent ID returns HTTP 404.

FR-EXAM-03: The system shall provide a filterable list of upcoming exams. Filters include batch, exam type, and room. Only exams with a date on or after the current date shall be returned.

Acceptance Criteria: GET `/admin/exams/list?batch=27&type=Midterm` returns only upcoming midterm exams for batch 27.

FR-EXAM-04: Students shall be able to view exam schedules relevant to their enrolled batch.

Acceptance Criteria: Student exam list endpoint returns exams matching the authenticated student's batch.

FR-ADMIT: Admission Hub

FR-ADMIT-01: The system shall provide a multi-step admission application form accessible to unauthenticated visitors. The form shall collect: personal information (name, email, date of birth, program, profile image), academic background (qualification, institution, field of study, graduation date, GPA), and document uploads (required document, academic transcript, recommendation letter, personal essay).

Acceptance Criteria: Submitting the complete multi-step form via POST `/guest/admission/apply` creates an `AdmissionForm` record and returns HTTP 201.

FR-ADMIT-02: The system shall record the submission timestamp (`form_given_on`) for every admission application.

Acceptance Criteria: Retrieved admission form records contain a non-null `form_given_on` field.

FR-ADMIT-03: The system shall allow administrators to list all submitted admission forms ordered by submission date (most recent first).

Acceptance Criteria: GET `/admin/admission/list` returns an array sorted by `form_given_on` descending.

FR-ADMIT-04: The system shall allow administrators to delete an admission form record by its ID.

Acceptance Criteria: DELETE `/admin/admission/delete/{id}` returns HTTP 200; missing record returns HTTP 404.

FR-ADMIT-05: The public Admission Hub shall display program details, course listings, syllabus information, and individual course details, all accessible without authentication.

Acceptance Criteria: Routes `/admission-hub`, `/program/:id`, `/program/:id/courses`, `/course/:id`, `/course/:id/syllabus` all render without a login token.

FR-COURSE: Course Management

FR-COURSE-01: The system shall allow a teacher to create a course with a title, description, semester, batch, type (Theory or Lab), optional image URL, other links, and one or more weekly schedule slots. The system shall auto-generate a unique 10-character alphanumeric enrolment passcode.

Acceptance Criteria: POST `/v1/courses/create` returns a course object containing a `code` field of length 10.

FR-COURSE-02: The system shall allow a student to enrol in a course by submitting the course passcode. The system shall validate the code and create the enrolment record.

Acceptance Criteria: POST `/v1/courses/enrol` with a valid `code` adds the student

to `student_courses`; invalid code returns HTTP 404.

FR-COURSE-03: The system shall allow students to list all their active (running) courses and separately list archived (completed) courses.

Acceptance Criteria: GET `/v1/courses/my-courses` returns courses where `running = true`; GET `/v1/courses/archived` returns courses where `running = false`.

FR-COURSE-04: The system shall allow a teacher to archive a course, setting `running = false`, preventing further student enrolment.

Acceptance Criteria: PUT `/v1/courses/archive/{id}` sets `running` to `false` and returns HTTP 200.

FR-COURSE-05: The system shall provide a classroom interface per course displaying course details, schedule, assignments, and resources.

Acceptance Criteria: Route `/teacher/classroom/{course_id}` renders course content; students access via `/student-dashboard`.

FR-ASSIGN: Assignment and Submission Management

FR-ASSIGN-01: The system shall allow a teacher to post an assignment of type `Homework` or `Resource` to a course, specifying title, description, maximum marks (optional), given date, due date (for homework), and one or more file links.

Acceptance Criteria: POST assignment endpoint returns HTTP 201; `type` field must be one of the two enum values, otherwise HTTP 422.

FR-ASSIGN-02: The system shall allow a student to submit a homework assignment by uploading one or more files before the due date.

Acceptance Criteria: POST submission endpoint creates a `submissions` record with `submitted_at` timestamp and HTTP 201.

FR-ASSIGN-03: The system shall allow a teacher to view all submissions for an assignment including the submitting student's details, submission time, and uploaded files.

Acceptance Criteria: Route `/assignments/{id}/submissions` renders a list of all submissions with student names.

FR-ASSIGN-04: The system shall allow a teacher to grade a submission by assigning a numeric marks value and marking it as checked.

Acceptance Criteria: PUT grade endpoint sets `marks` and `checked = true`; returns updated submission object.

FR-RES: Resource Hub

FR-RES-01: The system shall serve course materials (PDFs, slides, and other files) as static files via the `/resources` mount point. Files shall be accessible to authenticated users with the resource URL.

Acceptance Criteria: A file uploaded to the resource hub directory is accessible via

GET `/resources/{filename}` and returns the correct MIME type.

FR-RES-02: The system shall provide a student-facing Resource Hub page listing all downloadable materials associated with their enrolled courses.

Acceptance Criteria: Route `/resource-hub` renders a list of resources; each item provides a working download link.

FR-MEET: Departmental Meetings

FR-MEET-01: The system shall allow administrators to schedule a departmental meeting with a title, date and time, and a meeting URL (e.g., Google Meet, Zoom).

Acceptance Criteria: POST `/v1/meetings/create` returns HTTP 201 with the meeting object including all fields.

FR-MEET-02: The system shall allow administrators to archive a meeting, marking it as inactive without deleting the record.

Acceptance Criteria: PUT archive endpoint sets `is_archived = true`; the meeting no longer appears in the active list.

FR-MEET-03: The system shall allow any authenticated user to respond to a meeting invitation with a Yes or No RSVP. An existing RSVP for the same user and meeting shall be updated rather than duplicated.

Acceptance Criteria: POST RSVP endpoint with `response: "Yes"` creates or updates the record; GET RSVP count returns correct totals.

FR-MEET-04: The system shall display active (non-archived) meetings on the public Departmental Meetings page.

Acceptance Criteria: GET `/v1/meetings/list` returns only meetings where `is_archived = false`.

FR-EQUIP: Equipment Borrowing

FR-EQUIP-01: The system shall allow administrators to create equipment records in the inventory with a name, description, available quantity, and optional image URL.

Acceptance Criteria: POST `/admin/equipment/create` returns HTTP 201 with the equipment object.

FR-EQUIP-02: The system shall allow students to submit a borrow request for a piece of equipment, specifying the quantity and the start and end dates.

Acceptance Criteria: POST `/student/equipment/borrow` creates a `StudentEquipment` record with `returned = false`.

FR-EQUIP-03: The system shall allow administrators to mark borrowed equipment as returned, updating the returned flag and adjusting available inventory quantity.

Acceptance Criteria: PUT return endpoint sets `returned = true`; equipment's `quantity_available`

increases accordingly.

FR-EQUIP-04: Students shall be able to view their own current and past equipment borrow history.

Acceptance Criteria: GET `/student/equipment/my-orders` returns all `StudentEquipment` records for the authenticated student.

FR-FIN: Finance Management

FR-FIN-01: The system shall allow administrators to create finance events (fee items) with a title, amount, batch scope, and payment deadline.

Acceptance Criteria: POST `/v1/finance/events` returns HTTP 200 with the finance event; missing required fields return HTTP 422.

FR-FIN-02: The system shall allow administrators to update or delete existing finance events.

Acceptance Criteria: PUT `/v1/finance/update/event/{id}` updates all fields; DELETE returns HTTP 204.

FR-FIN-03: The system shall allow students to submit a payment record for a finance event by providing a transaction ID obtained from the mock payment gateway. The initial status shall be `pending`.

Acceptance Criteria: POST `/v1/finance/payments` creates a `StudentPayment` record with `status = "pending"` and HTTP 200.

FR-FIN-04: The system shall allow administrators to verify a student payment, updating the status to `paid` and recording the verification timestamp.

Acceptance Criteria: PUT `verify` endpoint sets `status = "paid"` and `verified_at` to the current timestamp.

FR-FIN-05: The system shall provide a mock payment gateway component that simulates a payment flow, generating a transaction ID for the student to submit.

Acceptance Criteria: The `MockPaymentGateway` component renders and produces a transaction ID string upon confirmation.

FR-FIN-06: The admin finance dashboard shall display all pending and paid payment records with student and event details.

Acceptance Criteria: Route `/admin-finance` (protected, admin role) lists all payment records with their status.

FR-TEACHER: Teacher Profile Management

FR-TEACHER-01: The system shall allow a teacher to view and update their profile including first name, last name, department, phone, title/position, biography, and profile image.

Acceptance Criteria: PUT `profile` update endpoint returns HTTP 200 with updated teacher record.

FR-TEACHER-02: The system shall allow a teacher to add, update, and delete academic profile links including personal website, Google Scholar, Academia, LinkedIn, Twitter, WhatsApp, and ResearchGate.

Acceptance Criteria: Updated links are persisted and returned in the teacher profile GET response.

FR-TEACHER-03: The system shall allow a teacher to add and remove research paper links associated with their profile.

Acceptance Criteria: POST research paper endpoint creates a **ResearchPaper** record; DELETE removes it by ID.

FR-TEACHER-04: The system shall display a teacher dashboard with summary cards for: total courses, upcoming schedule, recent activity log entries, and pending assignments.

Acceptance Criteria: GET `/v1/teacher/dashboard` returns all summary data in a single response.

FR-STUDENT: Student Dashboard and Attendance

FR-STUDENT-01: The system shall provide a student dashboard displaying: enrolled courses count, upcoming exams, recent notices, and current finance obligations.

Acceptance Criteria: Route `/student-dashboard` renders all summary sections after authentication with role **student**.

FR-STUDENT-02: The system shall allow administrators to record the monthly missing-class percentage for each student. Each record shall store the student ID, month name, year, and percentage.

Acceptance Criteria: POST attendance endpoint creates a **MissingClassOnMonth** record; duplicate month/year/student entries are rejected.

FR-STUDENT-03: The system shall allow students to view their attendance records broken down by month and year.

Acceptance Criteria: GET `/student/attendance` returns all **MissingClassOnMonth** records for the authenticated student.

FR-STUDENT-04: The system shall allow students to update their profile information including bio, phone, and profile image.

Acceptance Criteria: PUT `/student/settings/update` returns HTTP 200 with the updated student profile.

FR-ADMIN: Admin Dashboard

FR-ADMIN-01: The system shall provide an admin dashboard with navigation to: notice management, event management, exam scheduling, admission hub, equipment inventory, finance management, and payment verification.

Acceptance Criteria: Route `/admin-dashboard` renders all management panels for authenticated admin users.

FR-ADMIN-02: The system shall log teacher activities (e.g., creating a course, posting an assignment) with a message and timestamp, accessible to administrators.

Acceptance Criteria: Activity log records are created automatically on relevant teacher actions and are queryable.

Chapter 6

User Stories and Use Cases

6.1 User Stories

Authentication & Access

- As a **new user**, I want to register with my university email and password so that I can access the portal with my appropriate role.
- As a **registered user**, I want to log in with my email and password and receive a session token so that I can access protected features.
- As an **authenticated user**, I want to change my password securely so that I can maintain account safety.

Visitor Stories

- As a **visitor**, I want to browse the departmental home page so that I can learn about CSEDU without creating an account.
- As a **prospective student**, I want to view the People Directory to see faculty profiles and contact information.
- As a **prospective applicant**, I want to access the Admission Hub and fill out a multi-step application form with document uploads.
- As a **visitor**, I want to view upcoming departmental events so that I can decide whether to attend or register.

Student Stories

- As a **student**, I want to enrol in a course using a passcode so that I can access its materials and assignments.
- As a **student**, I want to view all my enrolled courses and their schedules in one place so that I can plan my week.
- As a **student**, I want to download course resources and submit homework assignments before their due dates.
- As a **student**, I want to view my upcoming exam schedule filtered by my batch so that I can prepare adequately.

- As a **student**, I want to see my finance obligations and submit payment records via the mock gateway.
- As a **student**, I want to borrow lab equipment by specifying quantities and dates so that I can use it for my coursework.
- As a **student**, I want to view my monthly attendance report so that I can track my class participation.
- As a **student**, I want to RSVP to departmental meetings so that I can confirm my attendance.

Teacher Stories

- As a **teacher**, I want to create a course with a schedule and auto-generated passcode so that students can enrol.
- As a **teacher**, I want to post assignments and resources to a course so that students can access them.
- As a **teacher**, I want to view and grade student submissions so that I can provide timely feedback.
- As a **teacher**, I want to manage my academic profile, bio, and research papers so that my professional information is publicly visible.
- As a **teacher**, I want to view my dashboard summary (courses, upcoming schedule, activity log) so that I have a quick overview of my responsibilities.
- As a **teacher**, I want to archive completed courses so that students can no longer enrol but can still access archived materials.

Admin Stories

- As an **admin**, I want to create and manage notices with batch filters and attachments so that the right audience receives the right information.
- As an **admin**, I want to schedule and manage departmental events so that students and visitors are informed of upcoming activities.
- As an **admin**, I want to create exam records with batch, type, room, and date so that students can view their upcoming exam schedule.
- As an **admin**, I want to review and delete submitted admission forms so that I can manage the applicant pipeline.
- As an **admin**, I want to manage the equipment inventory and mark returned items so that availability is always accurate.
- As an **admin**, I want to create fee events and verify student payments so that the department's financial records are up to date.
- As an **admin**, I want to schedule departmental meetings and archive them after they conclude.
- As an **admin**, I want to record monthly attendance missing percentages for each student so that attendance data is tracked digitally.

6.2 Critical Use Cases

UC-01: Student Course Enrolment

Attribute	Detail
Actor	Student
Precondition	Student is authenticated; teacher has created a course with a pass-code
Main Flow	Student navigates to Enrol Course; enters the 10-character pass-code; system validates code and creates enrolment record; student is redirected to course dashboard
Alternative Flow	Invalid passcode – system returns error “Course not found”; student re-enters or contacts teacher
Postcondition	Student appears in course’s enrolment list; course appears in student’s My Courses

Table 6.1: UC-01: Student Course Enrolment

UC-02: Admin Creates Exam Schedule

Attribute	Detail
Actor	Admin
Precondition	Admin is authenticated
Main Flow	Admin navigates to Exam Management; fills in subject, date/time, duration, batch, room, and type (Midterm/Final); submits form; system creates exam record
Alternative Flow	Missing required field – system returns HTTP 422 with field-level error message
Postcondition	Exam record is persisted; students of the specified batch can view the exam in their schedule

Table 6.2: UC-02: Admin Creates Exam Schedule

UC-03: Student Submits Finance Payment

Attribute	Detail
Actor	Student
Precondition	Admin has created a finance event for the student's batch; student is authenticated
Main Flow	Student views pending fee event; opens Mock Payment Gateway; completes simulated payment; receives transaction ID; submits transaction ID to the portal; record is created with status pending
Alternative Flow	Student submits duplicate payment – system rejects with HTTP 400
Postcondition	StudentPayment record exists with status = pending ; admin can verify

Table 6.3: UC-03: Student Submits Finance Payment

UC-04: Visitor Submits Admission Application

Attribute	Detail
Actor	Visitor (unauthenticated)
Precondition	None
Main Flow	Visitor navigates to /apply ; completes Step 1 (personal info); Step 2 (academic background); Step 3 (document uploads); submits; system creates AdmissionForm record
Alternative Flow	Incomplete step – form wizard prevents navigation to next step until all required fields are filled
Postcondition	Admission form is stored with timestamp; visible to admin in Admissions list

Table 6.4: UC-04: Visitor Submits Admission Application

Chapter 7

Data Requirements

7.1 Core Entities

Entity	Purpose
User	Core authentication record: email, hashed password, role
Student	Student profile linked to a user; stores batch, bio, phone
Teacher	Teacher profile linked to a user; stores bio, links, department
Course	Course entity with code/passcode, semester, batch, type (Theory/Lab)
StudentCourse	Many-to-many enrolment junction
Schedule	Weekly class schedule linked to course and teacher
Assignment	Homework or Resource attached to a course
Submission	Student submission for an assignment with file links and marks
ActivityLog	Teacher action log with message and timestamp
ResearchPaper	Research paper links associated with a teacher
Meeting	Departmental meeting with URL, datetime, archival flag
RSVP	Meeting RSVP records (Yes/No) per user
Notice	Notices with batch scope, source enum, attachments
Event	Departmental events with dates, location, media URLs
Exam	Exam records (Midterm/Final) with batch, room, duration
Equipment	Lab equipment inventory
StudentEquipment	Borrow transaction linking student and equipment
FinanceEvent	Fee payment events with batch scope and deadline
StudentPayment	Student payment record with transaction ID and status
AdmissionForm	Multi-step admission application with uploaded document paths
MissingClassOnMonth	Monthly attendance gap percentage per student

Table 7.1: Core Data Entities

7.2 Data Dictionary

Entity	Field	Type	Description
User	id	Integer (PK)	Auto-increment primary key
User	email	String (unique)	University email address
User	hashed_password	String	Bcrypt-hashed password
User	role	Enum	student teacher admin
Student	user_id	Integer (FK)	References users.id
Student	batch	Integer	Academic cohort number
Student	bio	Text	Student biography
Student	phone	String	Contact number
Course	id	Integer (PK)	Auto-increment primary key
Course	title	String	Course title
Course	code	String(10)	Auto-generated enrolment passcode
Course	semester	String	e.g., Spring 2026
Course	batch	Integer	Target student batch
Course	type	Enum	Theory Lab
Course	running	Boolean	True if active, False if archived
Exam	id	Integer (PK)	Auto-increment primary key
Exam	subject	String	Subject name
Exam	date_time	DateTime	Scheduled date and time
Exam	duration	Integer	Duration in minutes
Exam	batch	Integer	Target batch number
Exam	room	String	Room identifier
Exam	type	Enum	Midterm Final
FinanceEvent	id	Integer (PK)	Auto-increment primary key
FinanceEvent	title	String	Fee item name
FinanceEvent	amount	Float	Amount in BDT
FinanceEvent	batch	Integer	Target student batch
FinanceEvent	deadline	Date	Payment deadline
StudentPayment	id	Integer (PK)	Auto-increment primary key
StudentPayment	student_id	Integer (FK)	References students
StudentPayment	event_id	Integer (FK)	References finance_events
StudentPayment	transaction_id	String	Mock gateway transaction ID
StudentPayment	status	Enum	pending paid
StudentPayment	verified_at	DateTime	Timestamp of admin verification

Table 7.2: Data Dictionary (Selected Key Fields)

7.3 Data Model

The relational model implements the following key associations:

- $\text{users} \leftarrow 1:1 \rightarrow \text{students} / \text{teachers}$
- $\text{students} \leftarrow M:N \rightarrow \text{courses}$ (via `student_courses`)
- $\text{teachers} \leftarrow 1:M \rightarrow \text{courses}, \text{schedules}, \text{research_papers}$
- $\text{courses} \leftarrow 1:M \rightarrow \text{schedules}, \text{assignments}$
- $\text{assignments} \leftarrow 1:M \rightarrow \text{submissions}$
- $\text{students} \leftarrow 1:M \rightarrow \text{submissions}, \text{student_equipments}, \text{student_payments}, \text{missing_classes}$
- $\text{meetings} \leftarrow 1:M \rightarrow \text{rsvps}$
- $\text{finance_events} \leftarrow 1:M \rightarrow \text{student_payments}$
- $\text{equipments} \leftarrow 1:M \rightarrow \text{student_equipments}$

Technology Stack:

Layer	Technology
Backend Framework	FastAPI (Python 3.12)
ORM	SQLAlchemy with Alembic migrations
Authentication	JWT (python-jose), bcrypt password hashing
Frontend Framework	React 18 + Vite, React Router v6
Styling	Tailwind CSS
File Storage	Local static file server mounted at <code>/resources</code>
Database	Relational (SQLite for development; PostgreSQL-ready)

Table 7.3: Implemented Technology Stack

Chapter 8

UI and UX Requirements

8.1 Wireframes

The following describes the layout and key UI elements for each major screen in CSE SmartPortal.

Public Home Page (/): A full-width video banner with the department name overlay; navigation bar with links to People, Events, Notice Board, Meetings, and Admission Hub; footer with departmental contact information.

Student Dashboard (/student-dashboard): Summary cards showing enrolled courses count, upcoming exams, recent notices count, and pending finance obligations; quick-access links to My Courses, Resource Hub, and Finance.

Teacher Dashboard (/teacher-dashboard): Cards for total courses, upcoming scheduled classes, recent activity log entries, and pending assignments awaiting grading; course list with classroom links.

Admin Dashboard (/admin-dashboard): Navigation grid to all management panels: Notice Management, Event Management, Exam Scheduling, Admission Hub, Equipment Inventory, Finance Management, and Meeting Scheduling.

Admission Application (/apply): A clearly labelled 3-step wizard with a progress bar:

1. Step 1 – Personal Information (name, email, date of birth, program, profile image upload)
2. Step 2 – Academic Background (qualification, institution, field, graduation date, GPA)
3. Step 3 – Document Uploads (required document, academic transcript, recommendation letter, personal essay)

Course Classroom (/teacher/classroom/:id): Tabbed interface with tabs for: Overview (schedule, description), Assignments & Resources (list with post button for teacher), and Submissions (assignment-specific list for teacher view).

8.2 UX and Accessibility Baseline

- **NFR-USE-01:** The user interface shall be responsive and usable on screen widths from 375px (mobile) to 1920px (desktop), implemented via Tailwind CSS responsive utilities.
- **NFR-USE-02:** Error messages returned by the API shall be human-readable and descriptive; the frontend shall surface these messages inline on the relevant form field.
- **NFR-USE-03:** Form validations shall provide inline feedback before API submission to prevent unnecessary network requests.
- **NFR-USE-04:** The admission application shall be presented as a clearly labelled multi-step wizard with progress indication so that applicants know how many steps remain.
- **NFR-USE-05:** All interactive controls (buttons, links, form inputs) shall have visible focus indicators to support keyboard navigation.
- **NFR-USE-06:** Loading states shall be shown with a visual indicator (spinner or skeleton) while asynchronous API requests are in progress.

Chapter 9

Non-Functional Requirements

Performance

- **NFR-PERF-01:** API endpoints shall respond within 500 ms for read operations and within 1 second for write operations under a load of up to 50 concurrent users.
- **NFR-PERF-02:** Frontend pages shall achieve a First Contentful Paint (FCP) of under 3 seconds on a standard broadband connection.
- **NFR-PERF-03:** File upload responses (resource hub, profile images) shall complete within 5 seconds for files up to 10 MB.

Security

- **NFR-SEC-01:** All passwords shall be hashed using bcrypt before storage; plain-text passwords shall never be persisted.
- **NFR-SEC-02:** All API communication shall occur over HTTPS in production.
- **NFR-SEC-03:** JWT tokens shall expire after a configurable period (default 24 hours); expired tokens shall return HTTP 401.
- **NFR-SEC-04:** The CORS policy shall be configured to allow only the designated frontend origin in production (currently * in development only).
- **NFR-SEC-05:** Role-based access control shall be enforced at the API layer; frontend route guards provide a secondary UX layer only.
- **NFR-SEC-06:** Sensitive configuration (secret keys, database URLs) shall be stored in environment variables, never hard-coded.

Reliability

- **NFR-REL-01:** The system shall target 99% uptime during active academic semesters.
- **NFR-REL-02:** Database transactions shall be atomic; partial writes on failure shall be rolled back.
- **NFR-REL-03:** The backend shall return structured JSON error responses (not raw stack traces) for all 4xx and 5xx errors.

Maintainability

- **NFR-MAINT-01:** The codebase shall be maintained in a public Git repository with a clear branching strategy and descriptive commit messages.
- **NFR-MAINT-02:** Backend code shall follow a modular structure separating models, schemas, and API routers per feature domain (as implemented in `app/Sami` and `app/Faiak` sub-packages).
- **NFR-MAINT-03:** Frontend components shall be organised by actor role (`/Admin`, `/Teacher`, `/Student`, `/pages`) for navigability.
- **NFR-MAINT-04:** All database schema changes shall be managed via SQLAlchemy declarative models; table creation is automated via `Base.metadata.create_all`.

Scalability

- **NFR-SCALE-01:** The backend shall be stateless (JWT-based), allowing horizontal scaling behind a load balancer without session affinity.
- **NFR-SCALE-02:** The database layer shall be replaceable from SQLite to PostgreSQL by updating the `DATABASE_URL` environment variable without code changes.

Chapter 10

Risks and Open Questions

10.1 Risk Register

Risk ID	Category	Severity	Description	Mitigation
R-01	Security	High	JWT secret key exposed in source code	Use environment variables; enforce <code>.gitignore</code> on <code>.env</code> files
R-02	Data Loss	High	No automated database backup mechanism	Schedule daily SQLite file backup or migrate to managed PostgreSQL with point-in-time recovery
R-03	File Storage	Medium	Local filesystem file storage does not scale; files lost on server restart in some environments	Migrate to object storage (S3-compatible) in production
R-04	CORS	Medium	Wildcard CORS (*) in development may be accidentally deployed to production	Enforce origin-specific CORS before production deployment
R-05	Scope Creep	Medium	Additional feature requests from department may delay delivery	Freeze requirements at SRS v2.0; document new requests as v3.0 candidates
R-06	Performance	Low	No pagination on list endpoints; large datasets may cause slow responses	Add cursor-based or offset pagination to all list endpoints
R-07	Usability	Low	Multi-step admission form may lose data if user navigates away	Implement local state persistence (e.g., session storage) across wizard steps

Table 10.1: Risk Register

10.2 Open Questions

1. **Real Payment Integration:** Will the department require integration with a real Bangladeshi payment gateway (Bkash, Nagad, SSLCommerz) in future versions? If so, the finance module architecture must be extended.
2. **Email Notifications:** Should the system send email notifications for new notices, exam schedules, or meeting invitations? This would require an SMTP integration

not currently in scope.

3. **Admission Approval Workflow:** The current implementation allows admins to list and delete applications but does not support status transitions (Pending → Shortlisted → Accepted/Rejected). Should this be added?
4. **Data Retention Policy:** How long should archived courses, old exam records, and payment history be retained? Is there a data purge schedule?
5. **Accessibility Standard:** Should the frontend comply with WCAG 2.1 AA? If so, a formal accessibility audit and remediation sprint should be planned.
6. **Multi-Department Support:** Will the portal be extended to other departments in the university? If yes, a multi-tenant architecture is required from the start.

Chapter 11

Testability and Traceability

11.1 Requirement Traceability

Requirement ID	Backend File(s)	Frontend File(s)
FR-AUTH-01-05	userApi.py, auth.py	Login.jsx, SignUp.jsx, RequireAuth.jsx
FR-PUBLIC-01-06	—	HomePage.jsx, PeopleDirectory.jsx, NoticeBoard.jsx
FR-NOTICE-01-04	AdminNoticeApi.py, StudentNoticeApi.py	AdminNotice.jsx, StudentNotice.jsx
FR-EVENT-01-04	AdminEventApi.py, StudentEventApi.py	AdminEvent.jsx, StudentEventShow.jsx
FR-EXAM-01-04	AdminExamApi.py, StudentExamApi.py	AdminExam.jsx
FR-ADMIT-01-05	AdminAdmissionHubApi.py, GuestAdmissionHubApi.py	AdmissionHub.jsx, ApplicationForm.jsx
FR-COURSE-01-05	courseApi.py, StudentMyClassesApi.py	MyCourses.jsx, EnrollCourse.jsx
FR-ASSIGN-01-04	StudentAssignmentApi.py, TeacherMyClassesApi.py	TeacherClassroom.jsx, SubmissionList.jsx
FR-RES-01-02	main.py (static mount)	ResourceHub.jsx
FR-MEET-01-04	meetingApi.py	DepartmentalMeetings.jsx, AdminMeeting.jsx
FR-EQUIP-01-04	AdminEquipmentApi.py, StudentEquipmentApi.py	AdminEquipment.jsx
FR-FIN-01-06	financeApi.py, AdminFinance.py, StudentFinanceApi.py	AdminFinance.jsx, Finance.jsx, MockPaymentGateway.jsx
FR-TEACHER-01-04	teacherProfileApi.py, teacherDashboardApi.py	TeacherDashboard.jsx, TeacherSettings.jsx

Requirement ID	Backend File(s)	Frontend File(s)
FR-STUDENT-01-04	StudentDashboardApi.py, StudentSettingsApi.py	StudentDashboard.jsx, Settings.jsx
FR-ADMIN-01-02	All admin APIs	AdminDashboard.jsx

Table 11.1: Requirements Traceability Matrix

11.2 Initial Manual Test Plan

Test ID	Requirement	Test Steps	Expected Result
T-01	FR-AUTH-01	POST /v1/auth/register with valid email, password, role=student	HTTP 200; user object returned; student profile created
T-02	FR-AUTH-01	POST /v1/auth/register with duplicate email	HTTP 400 returned
T-03	FR-AUTH-02	POST /v1/auth/login with valid credentials	HTTP 200; access_token returned
T-04	FR-AUTH-02	POST /v1/auth/login with wrong password	HTTP 401 returned
T-05	FR-AUTH-04	GET admin endpoint with student JWT	HTTP 403 returned
T-06	FR-EXAM-01	POST /admin/exams/create with all required fields	HTTP 201; exam record created
T-07	FR-EXAM-01	POST /admin/exams/create with missing subject field	HTTP 422 returned
T-08	FR-EXAM-03	GET /admin/exams/list?batch=27&type=Midterm	Only upcoming midterm exams returned
T-09	FR-ADMIT-01	POST /guest/admission/apply with complete 3-step form data	HTTP 201; AdmissionForm record created
T-10	FR-COURSE-01	POST /v1/courses/create by authenticated teacher	HTTP 200; course object with 10-character code returned
T-11	FR-COURSE-02	POST /v1/courses/enrol with valid passcode	Student added to course; HTTP 200

Test ID	Requirement	Test Steps	Expected Result
T-12	FR-COURSE-02	POST /v1/courses/enrol with invalid passcode	HTTP 404 returned
T-13	FR-FIN-03	POST /v1/finance/payments with valid transaction ID	Record created with status=pending
T-14	FR-FIN-04	PUT verify endpoint by admin	status updated to paid; verified_at set
T-15	FR-MEET-03	POST RSVP with response=Yes; POST again with response=No	First creates record; second updates (no duplicate)
T-16	FR-EQUIP-02	POST borrow request with valid equipment ID, quantity, dates	StudentEquipment record created with returned=false
T-17	FR-ASSIGN-02	POST submission before due date	Submission record created with submitted_at timestamp
T-18	FR-ASSIGN-04	PUT grade endpoint with marks value	marks set; checked=true; HTTP 200 returned
T-19	FR-AUTH-05	Navigate to /admin-dashboard without admin token in browser	Redirected to /login
T-20	NFR-SEC-01	Inspect database record for user password	Bcrypt hash stored; no plaintext

Table 11.2: Initial Manual Test Plan

Chapter 12

Lab 2 Submission Checklist

12.1 Lab 2 Submission Checklist

- ☐ Problem statement and scope are clear.
- ☐ Minimum 12 user stories with acceptance criteria. *(16 provided)*
- ☐ Functional requirements table is complete. *(22 requirements)*
- ☐ Data model and data dictionary are attached.
- ☐ Minimum 5 wireframes or flow screens included. *(attach separately as image files)*
- ☐ NFR section is completed. *(15 NFRs)*
- ☐ Risks and open questions are documented.
- ☐ Initial manual test plan (15 cases) is included. *(17 test cases provided)*

Appendix A

API Route Summary

Prefix	Module	Description
/v1/auth	Auth	Register, login, password change
/v1/courses	Courses	Create, enrol, list, archive
/v1/teacher	Teacher Dashboard	Dashboard summary, activity logs
/v1/teacher/profile	Teacher Profile	Profile CRUD, research papers
/v1/meetings	Meetings	Create, list, archive, RSVP
/v1/finance	Finance	Fee events, student payments
/student/dashboard	Student Dashboard	Summary data
/student/notice	Notices	Student notice list
/student/event	Events	Student event list
/student/exam	Exams	Student exam list
/student/equipment	Equipment	Borrow, view orders
/student/assignment	Assignments	View, submit
/student/classes	My Classes	Enrolled course list
/student/settings	Settings	Profile update
/student/finance	Finance	View events, submit payment
/teacher/classes	Teacher Classes	Course management
/admin/notice	Admin Notices	Create, delete
/admin/events	Admin Events	CRUD events
/admin/exams	Admin Exams	CRUD exams
/admin/equipment	Admin Equipment	Inventory management
/admin/admission	Admin Admissions	List, delete forms
/guest/admission	Guest Admission	Submit application
/utility	Utility	Upload, misc helpers

Table A.1: Implemented API Route Summary

Appendix B

Frontend Route Map

Route	Component	Access
/	HomePage	Public
/login	Login	Public
/signup	SignUp	Public
/people	PeopleDirectory	Public
/chairman	AboutChairman	Public
/meetings	DepartmentalMeetings	Public
/notice-board	NoticeBoard	Public
/studentEventShow	StudentEventShow	Public
/admission-hub	AdmissionHub	Public
/apply	ApplicationForm	Public
/student-dashboard	Dashboard	Student
/my-courses	MyCourses	Student
/enroll-course	EnrollCourse	Student
/archived-courses	ArchivedCourses	Student
/resource-hub	ResourceHub	Student
/student-notice	StudentNotice	Student
/finance	Finance	Student
/settingspage	SettingsPage	Student
/teacher-dashboard	TeacherDashboard	Teacher
/teacher/classroom/:id	TeacherClassroom	Teacher
/assignments/:id/submissions	SubmissionList	Teacher
/admin-dashboard	AdminDashboard	Admin
/admin-finance	AdminFinance	Admin

Table B.1: Frontend Route Map

Appendix C

References

1. IEEE Recommended Practice for Software Requirements Specifications (IEEE 830-1998)
2. CSEDU Project Tracks Guide
3. FastAPI Official Documentation – <https://fastapi.tiangolo.com>
4. React Official Documentation – <https://react.dev>
5. SQLAlchemy ORM Documentation – <https://docs.sqlalchemy.org>
6. CSE 4113 Internet Programming Lab Course Materials